

Quy hoạch động (Dynamic Programming)

PHỤ THUỘC C/13 cho lời giải vét cạn làm điểm xuất phát; C/15 cho ranh giới “khi nào tham lam đủ”; A/04 cho lập luận cắt-dán chứng minh cấu trúc con tối ưu.

TÓM TẮT

Quy hoạch động là kỹ thuật quan trọng nhất của phần C và cũng là kỹ thuật bị dạy tệ nhất. Nó không phải một danh sách bài mẫu cần thuộc; nó là một quy trình năm bước — định nghĩa trạng thái, viết công thức chuyển, xác định cơ sở, chọn thứ tự tính, đọc đáp án — trong đó bước một chiếm chín phần mười độ khó. Chương đi qua sáu họ trạng thái hay gặp (một chiều, ba lô, hai chuỗi, khoảng, trên cây, mặt nạ bit), cách giảm bộ nhớ, cách truy vết lời giải, và bốn kỹ thuật tăng tốc khi số trạng thái nhân số chuyển quá lớn. Nếu chỉ nhớ một điều: trạng thái phải *đủ* — biết trạng thái thì phần tương lai không cần biết gì thêm về quá khứ — và mọi lỗi quy hoạch động đều quy về việc vi phạm đúng câu này.

KIẾN THỨC CẦN CÓ TRƯỚC

- **trạng thái (state)** — bản tóm tắt của quá khứ, đủ để quyết định tương lai. Nếu hai lịch sử khác nhau dẫn tới cùng trạng thái thì phần còn lại của bài toán là như nhau.
- **cấu trúc con tối ưu** — lời giải tối ưu của bài lớn chứa lời giải tối ưu của các bài con; chứng minh bằng lập luận cắt-dán (A/04).
- **bài toán con chồng lặp** — các nhánh khác nhau của cây đệ quy gặp lại cùng bài toán con — đây là lý do việc ghi nhớ có tác dụng.
- **ghi nhớ (memoization)** — cách viết từ trên xuống: hàm đệ quy lưu kết quả đã tính vào bảng, lần sau tra thay vì tính lại.
- **bảng từ dưới lên (tabulation)** — cách viết lặp: điền bảng theo một thứ tự bảo đảm mọi phụ thuộc đã có sẵn.
- **mảng cuộn (rolling array)** — kỹ thuật giảm bộ nhớ: nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ cần giữ hai tầng.

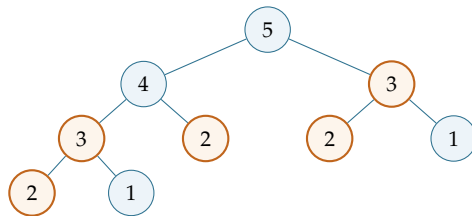
CÂU HỎI MỞ ĐẦU

- Q1. Quy hoạch động chỉ là đệ quy cộng một cái bảng. Vì sao nó lại khó tới vậy với hầu hết người học?
- Q2. Vì sao “chọn sai trạng thái” là lỗi phổ biến nhất, và làm sao biết trạng thái của mình *đủ* trước khi cài?
- Q3. Bài ba lô có lời giải $O(nW)$, nhìn thì đa thức. Vì sao bài ba lô vẫn được xếp vào NP-khó?
- Q4. Viết từ trên xuống (ghi nhớ) và từ dưới lên (điền bảng) cho cùng kết quả. Khi nào phải chọn cách nào?
- Q5. Khi số trạng thái đã tối thiểu mà vẫn quá chậm, còn hướng nào ngoài việc đổi thuật toán?

Người học quy hoạch động thường đi qua ba giai đoạn. Giai đoạn một: đọc lời giải thấy hiển nhiên, tự làm thì không nghĩ ra. Giai đoạn hai: nhận ra một số dạng quen thuộc và giải được các bài gần với chúng, nhưng gặp bài lạ thì lại đứng. Giai đoạn ba: nhìn bài mới và tự đặt ra được

trạng thái. Khoảng cách giữa giai đoạn hai và ba là khoảng cách giữa “thuộc bài mẫu” và “nắm phương pháp”, và chương này viết cho đúng khoảng cách đó.

Điểm khởi đầu phải là chương C/13: quy hoạch động không phải kỹ thuật độc lập mà là *vết cạn cộng trí nhớ*. Bạn viết lời giải đệ quy duyệt mọi khả năng — đúng nhưng chậm — rồi quan sát rằng rất nhiều nhánh khác nhau của cây đệ quy dẫn tới cùng một tình huống. Nếu tình huống đó mô tả được gọn thì lưu kết quả lại, và chi phí sụp từ hàm mũ xuống bằng *số tình huống phân biệt*. Cả chương này chỉ là các cách trả lời câu hỏi “tình huống được mô tả gọn bằng cái gì”.



Các nút tô hổ phách là cùng một bài toán con được tính lại nhiều lần. Ghi nhớ chúng biến cây thành đồ thị: chi phí sụp từ hàm mũ xuống *số trạng thái phân biệt*.

Hình 1.1. Cây đệ quy của một lời giải vét cạn. Quy hoạch động không phải kỹ thuật mới mà chỉ là quan sát rằng cây này có rất nhiều nút lặp lại. Toàn bộ độ khó nằm ở chỗ mô tả “nút nào giống nút nào” cho gọn — tức là ở việc chọn trạng thái.

1.1 Quy trình năm bước

Mọi lời giải quy hoạch động đều được viết theo cùng năm bước. Viết chúng ra thành lời *trước khi gõ code* là thói quen tạo ra khác biệt lớn nhất, đúng tinh thần chương A/04.

Bước 1 — định nghĩa trạng thái. Viết một câu hoàn chỉnh dạng “ $f[i][j]$ là ...”. Câu này phải rõ tới mức người khác đọc xong tính được ví dụ nhỏ bằng tay. Đây là bước chiếm chín phần mười độ khó, và mục 2 dành riêng cho nó.

Bước 2 — công thức chuyển. Từ trạng thái hiện tại, liệt kê *mọi* quyết định khả dĩ và trạng thái mà mỗi quyết định dẫn tới. Yêu cầu quan trọng: các nhánh phải *phủ hết* và tốt nhất là *không chồng nhau* (nếu đang đếm).

Bước 3 — cơ sở. Trạng thái nhỏ nhất và giá trị của nó. Rất nhiều lỗi nằm ở đây, đặc biệt việc phân biệt “không có cách nào” với “có đúng một cách rỗng”.

Bước 4 — thứ tự tính. Phải bảo đảm khi tính một trạng thái thì mọi trạng thái nó phụ thuộc đã xong. Nói cách khác, đồ thị phụ thuộc phải không có chu trình — và khi nó có chu trình thì bạn đang ở bài toán khác, cần Dijkstra hoặc giải hệ phương trình.

Bước 5 — đáp án và truy vết. Đáp án nằm ở trạng thái nào; nếu cần chỉ ra lời giải cụ thể chứ không chỉ giá trị, hãy lưu quyết định tại mỗi trạng thái hoặc dò ngược bằng cách so sánh giá trị.

1.2 Bước khó nhất: trạng thái phải đủ

Tiêu chuẩn duy nhất cho một trạng thái tốt: *biết trạng thái thì phần còn lại của bài toán hoàn toàn xác định, không cần biết thêm bất cứ điều gì về cách ta tới đó*. Trong ngôn ngữ xác suất, đây là tính chất Markov; trong ngôn ngữ chương A/04, đây chính là một bất biến.

Có ba cách phát hiện trạng thái thiếu, và cả ba đều rẻ hơn việc cài rồi mới sai.

Cách thứ nhất — thử kể lại tương lai. Đặt câu hỏi: “tôi đang ở trạng thái $f[i]$, tôi cần quyết định gì tiếp theo, và tôi có đủ thông tin để quyết định không?” Nếu câu trả lời là “còn tùy vào việc trước đó tôi đã chọn gì” thì trạng thái thiếu đúng cái đó, và cái đó phải trở thành một chiều mới.

Cách thứ hai — tìm hai lịch sử cùng trạng thái nhưng khác tương lai. Nếu dựng được hai cách đi tới cùng $f[i]$ mà phần tối ưu còn lại khác nhau thì trạng thái chắc chắn thiếu. Đây là phản ví dụ, và như chương A/04 đã nói, một phản ví dụ đủ để bác bỏ.

Cách thứ ba — kiểm tra cắt-dán. Nếu bạn không chứng minh được rằng thay một phần lời giải bằng lời giải tối ưu của bài con thì kết quả không tệ đi, thì cấu trúc con tối ưu không có — và chỗ chứng minh hỏng chỉ ra chiều còn thiếu. Ví dụ kinh điển: đường đi *dài nhất* không lập đỉnh trong đồ thị tổng quát; ghép hai đường dài nhất có thể trùng đỉnh, nên trạng thái phải mang theo tập đỉnh đã đi — và đó là lý do bài này chỉ giải được bằng mặt nạ bit với n nhỏ.

CẦN NHỚ

Ba câu hỏi để kiểm trạng thái trước khi cài: (i) tôi phát biểu được $f[\cdot]$ thành một câu đầy đủ không; (ii) từ trạng thái, tôi quyết định được bước tiếp theo mà không cần nhìn lại quá khứ chứ; (iii) hai lịch sử khác nhau tới cùng trạng thái có thật sự cho cùng tương lai không. Trả lời được cả ba thì phần còn lại là kỹ thuật.

1.3 Sáu họ trạng thái hay gặp

Thay vì học từng bài, hãy học sáu họ này — phần lớn bài quy hoạch động trong thi đấu và phỏng vấn là biến thể của chúng.

Họ 1 — một chiều theo vị trí. $f[i]$ là đáp án tối ưu xét tới phần tử i , thường kèm điều kiện “có dùng phần tử i ”. Đại diện: dãy con tăng dài nhất, chia dãy thành các đoạn, bài trộm nhà không lấy hai nhà liền kề. Dấu hiệu: quyết định tại mỗi vị trí chỉ phụ thuộc vài vị trí gần trước đó.

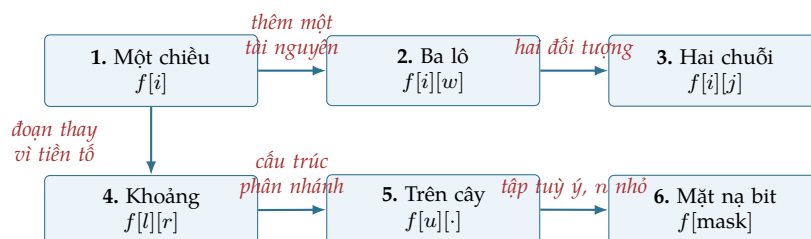
Họ 2 — ba lô: vị trí kèm một tài nguyên. $f[i][w]$ với w là lượng tài nguyên đã dùng. Đại diện: ba lô 0/1, đổi tiền, chia tập thành hai phần cân bằng. Dấu hiệu: có một đại lượng tích lũy bị giới hạn.

Họ 3 — hai chuỗi. $f[i][j]$ xét i ký tự đầu của chuỗi này và j ký tự đầu của chuỗi kia. Đại diện: dãy con chung dài nhất, khoảng cách chỉnh sửa, so khớp có ký tự đại diện. Dấu hiệu: bài toán so hai dãy với nhau.

Họ 4 — khoảng. $f[l][r]$ là đáp án cho đoạn $[l, r]$, tính theo độ dài tăng dần. Đại diện: nhân dãy ma trận, cắt gậy, chuỗi đối xứng, trò chơi lấy đồ ở hai đầu. Dấu hiệu: bài toán có cấu trúc lồng nhau, hoặc quyết định chia đoạn tại một điểm giữa.

Họ 5 — trên cây. $f[u][\cdot]$ là đáp án cho cây con gốc u ; tính từ lá lên. Đại diện: tập độc lập lớn nhất trên cây, phủ đỉnh, đường kính, đếm cách tô màu. Dấu hiệu: dữ liệu là cây, và điều kiện chỉ ràng buộc giữa cha và con.

Họ 6 — mặt nạ bit. $f[\text{mask}][\cdot]$ với mask mô tả tập đã dùng. Đại diện: người bán hàng, ghép cặp hoàn hảo, phủ hình. Dấu hiệu: $n \leq 20$ và bài yêu cầu “mỗi phần tử dùng đúng một lần”.



Hình 1.2. Sáu họ trạng thái, xếp theo mức độ thông tin phải mang theo. Đi từ trái sang phải và từ trên xuống dưới, trạng thái ngày càng nặng và số trạng thái ngày càng lớn. Nguyên tắc chọn: dùng họ *nhẹ nhất* còn đủ mô tả tương lai — cùng nguyên tắc “dùng cấu trúc yếu nhất còn đủ dùng” của phần B.

Ngoài sáu họ trên còn ba họ chuyên hơn, nên biết là có: *quy hoạch động trên chữ số* cho các bài đếm số trong khoảng thoả điều kiện; *quy hoạch động kỳ vọng* cho các bài xác suất, ở đó thứ tự tính đôi khi phải đi ngược hoặc phải giải hệ phương trình; và *quy hoạch động trò chơi*, ở đó giá trị của trạng thái được định nghĩa qua cực đại và cực tiểu xen kẽ.

1.4 Ba lô và câu hỏi “vì sao vẫn NP-khó”

Bài ba lô 0/1: n món, món i nặng w_i và có giá trị v_i ; chọn tập con nặng không quá W sao cho tổng giá trị lớn nhất. Trạng thái $f[i][w]$ = giá trị lớn nhất khi xét i món đầu và sức chứa còn w . Chuyển: lấy hoặc không lấy món i . Chi phí $O(nW)$.

Đây là chỗ nên dừng lại vì nó dạy một điều quan trọng về độ phức tạp. $O(nW)$ nhìn thì đa thức, nhưng W được ghi trong dữ liệu vào bằng $\log W$ bit. Nếu $W = 10^{18}$ thì kích thước dữ liệu vào chỉ tăng thêm vài chục bit trong khi chi phí tăng lên 10^{18} . Nói cách khác, thuật toán đa thức theo *giá trị* nhưng mũ theo *kích thước biểu diễn* — người ta gọi đó là **giả đa thức** (*pseudo-polynomial*). Bài ba lô vẫn là NP-khó, và điều đó hoàn toàn nhất quán với sự tồn tại của lời giải $O(nW)$. Chi tiết ở chương D/23.

Hệ quả thực hành: khi thấy trạng thái chứa một *giá trị* thay vì một chỉ số, hãy kiểm tra ngay giá trị đó lớn tới đâu. Nếu W lớn nhưng tổng giá trị nhỏ thì hãy đảo vai trò — đặt trạng thái theo giá trị và tối thiểu hoá khối lượng. Mẹo “đảo vai trò của mục tiêu và ràng buộc” này giải được rất nhiều bài trông bề bét.

1.5 Từ trên xuống hay từ dưới lên

Hai cách viết cho cùng kết quả, nhưng khác nhau ở bốn điểm thực dụng.

Ghi nhớ (từ trên xuống) gần với cách bạn nghĩ, nên viết nhanh và ít sai logic hơn; nó chỉ tính những trạng thái thực sự cần, điều rất quý khi không gian trạng thái thừa; và nó không đòi bạn nghĩ ra thứ tự tính. Nhược điểm: chi phí gọi hàm, nguy cơ tràn ngăn xếp khi độ sâu lớn, và khó áp dụng các tối ưu bộ nhớ.

Điền bảng (từ dưới lên) nhanh hơn theo hàng số, không tràn ngăn xếp, và cho phép giảm bộ nhớ bằng mảng cuộn. Nhược điểm: bạn phải tự xác định thứ tự tính đúng, và nó tính cả những trạng thái không cần.

Quy tắc thực dụng: nghĩ bằng ghi nhớ, và chuyển sang điền bảng khi cần tốc độ hoặc cần giảm bộ nhớ. Trong kỳ thi, nếu không gian trạng thái dày và thứ tự rõ ràng thì viết thẳng bảng.

Giảm bộ nhớ. Nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ giữ hai tầng — hoặc thậm chí một tầng nếu duyệt đúng chiều, như trong ba lô 0/1 khi duyệt w giảm dần. Kỹ thuật này biến $O(nW)$ bộ nhớ thành $O(W)$ và thường là thứ quyết định bài có chạy nổi hay không. Cái giá: bạn mất khả năng truy vết lời giải cụ thể, vì các tầng cũ đã bị ghi đè. Khi cần cả hai, dùng kỹ thuật chia để trị của Hirschberg cho họ hai chuỗi, hoặc lưu quyết định dưới dạng bit.

1.6 Bốn kỹ thuật tăng tốc

Chi phí quy hoạch động bằng *số trạng thái nhân chi phí mỗi chuyển*. Khi số trạng thái đã tối thiểu mà vẫn chậm, phải tấn công thừa số thứ hai. Bốn kỹ thuật, xếp theo mức độ hay gặp:

Hàng đợi đơn điệu. Khi chuyển có dạng “lấy cực trị của $f[j]$ với j trong một cửa sổ trượt”, dùng deque đơn điệu (B/07) để mỗi trạng thái tốn $O(1)$ khấu hao thay vì $O(k)$.

Cấu trúc dữ liệu cho truy vấn đoạn. Khi chuyển lấy cực trị hoặc tổng trên một đoạn tùy ý, dùng cây phân đoạn hoặc Fenwick (B/10) để giảm từ $O(n)$ xuống $O(\log n)$. Đây là cách tăng tốc dãy con tăng dài nhất từ $O(n^2)$ xuống $O(n \log n)$.

Bao lồi. Khi chuyển có dạng $f[i] = \min_j (f[j] + a_j \cdot b_i)$ — tức là mỗi j đóng góp một đường thẳng theo biến b_i — thì việc lấy cực tiểu trên tập đường thẳng chính là bài bao lồi, và có cấu trúc trả lời trong $O(\log n)$ hoặc $O(1)$ khi các hệ số đơn điệu.

Tối ưu chia để trị và tối ưu Knuth. Khi điểm chuyển tối ưu đơn điệu theo trạng thái — tức là tối ưu của trạng thái sau không bao giờ nằm bên trái tối ưu của trạng thái trước — ta tính các trạng thái theo kiểu chia để trị và giảm một thừa số n . Điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm.

Có một hướng thứ năm ít được nhắc nhưng rất mạnh: *giảm số trạng thái bằng một quan sát tham lam*. Nếu chứng minh được rằng một lớp lựa chọn không bao giờ tối ưu, hãy loại chúng khỏi không gian trạng thái. Đây là chỗ chương C/15 và chương này gặp nhau, và nó thường hiệu quả hơn mọi tối ưu kỹ thuật.

1.7 Vì sao điểm tối ưu lại đơn điệu: bất đẳng thức tứ giác

Kỹ thuật thứ tư ở mục trên có một câu bỏ lửng: “điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm”. Có một điều kiện đủ, phát biểu được trong một dòng, và biết nó thì bạn không còn phải đoán.

Xét lớp truy hồi rất hay gặp ở họ khoảng:

$$f(i, j) = w(i, j) + \min_{i < k < j} (f(i, k) + f(k, j)),$$

với w là chi phí gộp. Nói w thoả **bất đẳng thức tứ giác** (*quadrangle inequality*) nghĩa là với mọi $a \leq b \leq c \leq d$,

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Đọc bằng lời: *hai đoạn lồng nhau không đắt hơn hai đoạn cắt nhau*. Đây đúng là hình ảnh của lập luận đổi chỗ ở chương A/04 — nếu hai lựa chọn tối ưu “bắt chéo” nhau, ta gỡ chéo được mà không làm tệ đi — và chính nó kéo theo tính đơn điệu của điểm chia tối ưu.

Lịch sử của điều kiện này minh hoạ cách một quan sát riêng lẻ trở thành công cụ chung. Knuth[1] nhận ra tính đơn điệu ấy khi tính cây tìm kiếm nhị phân tối ưu và nhờ đó hạ $O(n^3)$ xuống $O(n^2)$; hơn mười năm sau, Yao[2] tách bất đẳng thức tứ giác ra khỏi bài toán cụ thể và chứng minh nó là điều kiện đủ cho cả lớp truy hồi trên. Từ chỗ “một mẹo cho một bài”, nó thành “một định lý cho một họ bài” — và đó là lý do đáng học nó ở dạng tổng quát thay vì học thuộc bài cây tìm kiếm.

Đi xa hơn một bước là thuật toán SMAWK[3], đặt theo chữ đầu của năm tác giả. Khi ma trận chi phí *đơn điệu toàn phần* — một điều kiện họ hàng với bất đẳng thức tứ giác — thì cực tiểu của mọi hàng tìm được trong thời gian *tuyến tính* theo kích thước ma trận, không có cả thừa số lôgarit. Chi tiết đáng chú ý: bài báo này ra đời trong hình học tính toán, cho bài tìm đỉnh xa nhất từ mỗi đỉnh của một đa giác lồi, rồi mới di cư sang quy hoạch động. Lại một lần nữa, cùng cấu trúc toán học xuất hiện ở hai miền không liên quan — đúng điều mà mục liên hệ ngang của mọi chương trong quyển này cố chỉ ra.

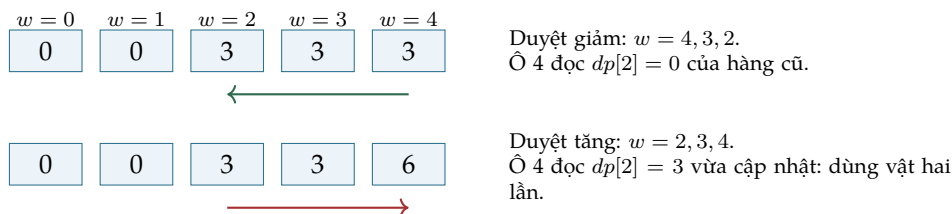
Còn một mảnh nữa hoàn thiện bức tranh, và nó trả lời câu hỏi “đã nén bộ nhớ thì truy vết thế nào” đã hẹn ở mục 5. Hirschberg[4] lấy họ hai chuỗi làm ví dụ: tính bảng từ trái sang tới cột giữa

và từ phải sang tới cột giữa, cộng hai kết quả lại thì biết lời giải tối ưu đi qua ô nào của cột giữa; cắt bài toán tại ô đó và đệ quy trên hai nửa. Tổng chi phí vẫn $O(nm)$ vì mỗi tầng đệ quy làm việc trên một nửa diện tích, còn bộ nhớ chỉ $O(\min(n, m))$. Ý tưởng “không lưu vết, mà tính lại có tổ chức” này dùng được cho nhiều họ trạng thái khác, không riêng hai chuỗi.

Lời khuyên thực hành khép lại mục: trước khi tin vào tính đơn điệu, hãy *kiểm* nó — viết một đoạn vết cạn nhỏ tính điểm chia tối ưu cho mọi (i, j) với n cỡ vài chục và kiểm tra trực tiếp bất đẳng thức tứ giác trên hàm w của bạn. Năm phút đó rẻ hơn nhiều so với việc gỡ một lời giải sai chỉ trên vài phần trăm bộ test (E/25).

1.8 Cùng một công thức, đổi chiều duyệt là đổi bài toán

Xét một vật duy nhất có trọng lượng 2, giá trị 3 và sức chứa 4. Với trạng thái “giá trị tốt nhất khi sức chứa không quá w ”, khởi tạo mọi ô bằng 0. Công thức nén là $dp[w] \leftarrow \max(dp[w], dp[w-2]+3)$. Hình 1.3 cho thấy chiều duyệt quyết định ô bên phải đọc giá trị của hàng cũ hay hàng mới.



Hình 1.3. Hai mảng sau khi xử lý cùng một vật. Hàng trên đúng cho ba lô 0/1; hàng dưới đúng cho ba lô không giới hạn. Giá trị 6 không phải lỗi cộng mà là lỗi phụ thuộc nếu đề chỉ cho lấy vật một lần.

Bảng 1.1. Ba dạng ba lô: cùng tài nguyên nhưng khác tập quyết định hợp lệ. Giả sử trọng lượng nguyên dương.

Dạng	Giới hạn mỗi vật	Cách duyệt / xử lý	Test phân biệt
0/1	Tối đa một lần	Duyệt sức chứa giảm, $O(nW)$	Một vật, $W = 2w_i$: không được lấy hai lần
Không giới hạn	Bao nhiêu lần cũng được	Duyệt sức chứa tăng, $O(nW)$	Một vật, $W = 2w_i$: được lấy hai lần
Có số lượng	Tối đa c_i lần	Tách nhị phân thành vật 0/1, $O(W \sum_i \log(c_i + 1))$	Sức chứa đủ cho $c_i + 1$ vật: vẫn chỉ lấy c_i

Bảng 1.1 giúp chọn biến thể trước khi nén bộ nhớ. Nếu trạng thái yêu cầu trọng lượng *đúng bằng* w , chỉ $dp[0] = 0$, các ô còn lại là không đạt được ($-\infty$ trong bài cực đại), và không cộng vào trạng thái không đạt được. Khi chưa chắc, giữ bảng hai chiều làm bản đối chiếu; chỉ nén sau khi xác định rõ mỗi phép đọc thuộc hàng nào.

1.9 Phản biện, nhược điểm và rủi ro triển khai

Dù quy hoạch động là một trong những công cụ đẹp nhất của khoa học máy tính, việc thần thánh hoá nó như một giải pháp vạn năng cho mọi bài toán tối ưu là một sai lầm nguy hiểm trong kỹ thuật. Để dùng nó một cách trưởng thành, cần nhìn thẳng vào ba phương diện: khi nào nó bị các phương pháp khác đánh bại, nó mang nhược điểm cấu trúc gì, và những lỗi im lặng nào chực chờ làm sập hệ thống khi chạy thật.

Phản biện: sự lạm dụng và ranh giới áp dụng. Quy hoạch động thực chất là một phép vét cạn có tổ chức: nó vẫn buộc phải ghé thăm toàn bộ các trạng thái khả dĩ. Nếu bài toán sở hữu cấu trúc matroid (C/15), thuật toán tham lam tìm ra nghiệm tối ưu chỉ bằng một lượt quét đơn giản với bộ nhớ gần như $O(1)$. Nếu hàm mục tiêu có tính lồi hoặc đơn điệu, tìm kiếm nhị phân trên đáp án (C/19) hoặc kỹ thuật hai con trỏ (B/06) cho tốc độ vượt trội gấp nhiều bậc mà không cần lưu trữ bảng trạng thái. Đặc biệt, khi bài toán chứa ràng buộc toàn cục mang tính phi Markov — ví dụ như chi phí bước sau phụ thuộc vào phương sai của toàn bộ chuỗi lựa chọn trước đó — cấu trúc con tối ưu sụp đổ hoàn toàn. Cố tình nhồi nhét toàn bộ lịch sử vào trạng thái (như dùng mặt nạ bit) sẽ đẩy chi phí lên hàm mũ, biến quy hoạch động thành một cách viết rườm rà của thuật toán quay lui thô thiển.

Nhược điểm cốt lõi về tài nguyên và phần cứng. Có ba cái giá cấu trúc luôn đi kèm quy hoạch động:

- *Lời nguyền số chiều (Curse of Dimensionality):* Mỗi ràng buộc phụ của bài toán đòi hỏi mở thêm một chiều trạng thái mới. Số lượng ô nhớ tăng theo tích số các chiều; chỉ cần thêm hai tham số phụ với miền giá trị cỡ 10^3 , bộ nhớ lập tức phình to từ megabyte thành gigabyte, vượt ngưỡng bộ nhớ của mọi máy thi đấu hay thiết bị nhúng.
- *Phá vỡ phân cấp bộ nhớ và tính bất khả thi của SIMD:* Khác với các phép nhân ma trận có thể sắp xếp khối để tối ưu cache L1/L2, truy cập bảng quy hoạch động thường nhảy cóc theo bước nhảy của dữ liệu (như $w - w_i$ trong ba lô) hoặc theo con trỏ cây, gây trượt cache liên tục. Quan trọng hơn, do bản chất của hệ thức truy hồi luôn chứa phụ thuộc dữ liệu theo vòng lặp (loop-carried dependency: bước sau bắt buộc phải đợi bước trước hoàn thành), các compiler hiện đại gần như bất lực trong việc tự động vector hoá (SIMD) hay phân tán tính toán ra hàng nghìn nhân GPU.
- *Mất mát thông tin khi nén bộ nhớ:* Kỹ thuật mảng cuộn cứu được bộ nhớ nhưng triệt tiêu toàn bộ đường dẫn quyết định. Trong công việc thực tế, một hệ thống chỉ đưa ra con số chi phí tối ưu mà không thể chỉ ra phương án hành động cụ thể là một hệ thống vô dụng. Khôi phục lại đường đi buộc phải trả giá bằng việc nhân đôi thời gian chạy qua thuật toán chia để trị kiểu Hirschberg hoặc cài đặt các cấu trúc bit phức tạp.

Rủi ro vận hành và các chế độ hồng im lặng. Trong môi trường sản xuất và thi đấu, các lỗi của quy hoạch động hiếm khi ném ra ngoại lệ rõ ràng, mà chúng âm thầm trả về kết quả sai:

- *Tràn số khi gán vô cùng (∞):* Để tìm cực tiểu, ta thường khởi tạo các trạng thái không hợp lệ bằng một giá trị vô cùng lớn. Nếu chọn giá trị này bất cẩn (ví dụ dùng $0x7fffffff$ trong số nguyên 32-bit có dấu), phép tính $f[j] + cost$ sẽ bị tràn số số học thành một số âm rất lớn. Thuật toán sẽ nhầm tưởng trạng thái cấm đó là một lời giải siêu tối ưu, làm sai lệch hoàn toàn kết quả toàn cục. Quy tắc an toàn: dùng hằng số vừa đủ lớn (như $0x3f3f3f3f$ cho 32-bit hoặc 10^{18} cho 64-bit) để phép cộng hai số vô cùng không bị tràn.
- *Tràn số trong các bài toán đếm cách:* Số cách thực hiện có xu hướng bùng nổ theo cấp số nhân. Quên lấy dư modulo ở chỉ một phép cộng dồn trung gian, hoặc cộng hai số đã lấy modulo mà không kiểm tra xem tổng có vượt giới hạn kiểu dữ liệu hay không, sẽ gây tràn số âm thầm.
- *Tràn ngăn xếp đệ quy (Stack Overflow):* Dùng đệ quy có nhớ (memoization) trên các đồ thị trạng thái có chuỗi dài (như chuỗi $n \geq 10^5$ phần tử) sẽ nhanh chóng vượt quá dung lượng call stack mặc định của hệ điều hành (thường chỉ 8 MB trên Linux), gây sập chương trình đột ngột (Segmentation Fault) ngay trên dữ liệu lớn.
- *Hiện tượng underflow trong quy hoạch động xác suất:* Trong các bài toán xử lý tín hiệu hay thị giác (thuật toán Viterbi, HMM), việc nhân dồn lặp các xác suất $p \in (0, 1)$ qua hàng trăm bước thời gian sẽ đẩy giá trị về số 0 dấu phẩy động (underflow). Lời giải bắt buộc trong thực tế là chuyển toàn bộ sang miền logarit ($\ln p$) và thay phép nhân xác suất bằng phép cộng, kết hợp hàm log-sum-exp để tránh mất mát độ chính xác số học.

CẠM BÃY

Đừng bao giờ tối ưu bộ nhớ bằng mảng cuộn khi chưa kiểm thử xong bản hai chiều hoàn chỉnh. Mảng cuộn nén bộ nhớ nhưng làm mờ ranh giới thời gian giữa trạng thái cũ và trạng thái mới, khiến việc gỡ lỗi logic chuyển trạng thái trở nên khó gấp bội. Hãy luôn giữ một bản đầy đủ để đối chiếu và kiểm chứng tính đúng trước khi tối ưu hoá sớm.

1.10 Gốc rễ: một cái tên được chọn để nguy trang

Quy hoạch động do Richard Bellman phát triển tại RAND Corporation trong thập niên 1950, và câu chuyện về cái tên của nó đáng kể lại vì nó nói nhiều về bối cảnh nghiên cứu thời đó. Trong hồi ký của mình, Bellman kể rằng ông làm việc dưới sự tài trợ của Không quân Mỹ, và vị bộ trưởng quốc phòng khi đó có ác cảm với từ “nghiên cứu” và đặc biệt với “toán học”. Ông cần một cái tên nghe có vẻ thực dụng và không thể phản đối. “Dynamic” nghe tích cực và có nghĩa thời gian; “programming” thời đó mang nghĩa lập kế hoạch, như trong “quy hoạch tuyến tính”. Kết quả là một cái tên gần như không mô tả gì về kỹ thuật — điều gây khó khăn cho người học suốt bảy chục năm sau.

Nội dung thật sự thì nghiêm túc hơn cái tên nhiều. Bellman đưa ra *nguyên lý tối ưu*: một chính sách tối ưu có tính chất là, dù trạng thái ban đầu và quyết định đầu tiên là gì, các quyết định còn lại phải tạo thành chính sách tối ưu cho trạng thái sinh ra bởi quyết định đầu. Phát biểu đó chính là cấu trúc con tối ưu, và phương trình Bellman đi kèm là công thức chuyển ở dạng tổng quát nhất. Điều đáng nói là nó ra đời cho các bài toán *điều khiển liên tục theo thời gian* — quản lý tồn kho, điều khiển tên lửa — chứ không phải cho bài tập tổ hợp; các ứng dụng rời rạc là hệ quả về sau.

Ba nhánh ứng dụng lớn ra đời gần như đồng thời trong thập niên 1960–70, mỗi nhánh từ một ngành khác nhau, và cả ba đều là quy hoạch động dù người phát minh không luôn dùng từ đó. Trong *viễn thông*, Viterbi công bố năm 1967 thuật toán giải mã cho mã chập, về sau trở thành thuật toán chuẩn để tìm chuỗi trạng thái khả dĩ nhất của mô hình Markov ẩn — nền của nhận dạng tiếng nói suốt nhiều thập kỷ. Trong *tin sinh học*, Needleman và Wunsch công bố năm 1970 thuật toán căn chỉnh hai chuỗi protein — chính là bài khoảng cách chỉnh sửa với ma trận điểm số sinh học. Trong *ngôn ngữ học tính toán*, thuật toán phân tích cú pháp CYK dùng quy hoạch động trên khoảng để phân tích câu.

Sự trùng hợp này không phải ngẫu nhiên. Cả ba bài toán đều có cùng cấu trúc: một chuỗi quyết định, mỗi quyết định phụ thuộc trạng thái tóm tắt được, và mục tiêu cộng dồn. Bất cứ khi nào ba điều kiện đó xuất hiện — ở bất kỳ ngành nào — quy hoạch động sẽ được phát minh lại.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Bellman (RAND)	Điều khiển tối ưu nhiều giai đoạn: tồn kho, phân bổ tài nguyên, dẫn đường.	Nguyên lý tối ưu và phương trình Bellman; cái tên “dynamic programming” được chọn để tránh bị cắt tài trợ vì nghe giống “toán học”.
1962 — Held & Karp	Bài người bán hàng với n vừa phải.	Quy hoạch động trên mặt nạ bit, $O(2^n n^2)$ — vẫn là thuật toán chính xác tốt nhất đã biết cho bài này.
1965–70 — Levenshtein; Needleman & Wunsch	So sánh chuỗi: mã sửa lỗi, rồi căn chỉnh chuỗi protein.	Khoảng cách chỉnh sửa; nền của cả kiểm tra chính tả lẫn tin sinh học hiện đại.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1967 — Viterbi	Giải mã mã chập trong truyền tin có nhiễu.	Thuật toán Viterbi; về sau thành công cụ chuẩn cho mô hình Markov ẩn trong nhận dạng tiếng nói và xử lý tín hiệu.
Thập niên 1960 — CYK	Phân tích cú pháp câu theo văn phạm phi ngữ cảnh.	Quy hoạch động trên khoảng; cùng khuôn với bài nhân dây ma trận.
Thập niên 1980–nay — học tăng cường	Ra quyết định tuần tự khi mô hình môi trường không biết trước.	Phương trình Bellman trở thành nền của học tăng cường; “hàm giá trị” chính là bảng quy hoạch động, chỉ khác là nó được <i>học</i> thay vì được điền.

1.11 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Học tăng cường	Phương trình Bellman, hàm giá trị, lặp giá trị	Cùng một phương trình. Khác: ở đây bảng được điền bằng cách duyệt hết trạng thái, ở đó hàm giá trị được <i>xấp xỉ</i> bằng mạng nơ-ron vì không gian trạng thái quá lớn để liệt kê.
Lý thuyết điều khiển	Điều khiển tối ưu, bộ điều khiển toàn phương tuyến tính	Bản liên tục của cùng nguyên lý; “trạng thái” ở đây đúng nghĩa trạng thái vật lý. Điều kiện Markov trong điều khiển tương ứng chính xác với yêu cầu “trạng thái phải đủ” ở mục 2.
Tin sinh học	Căn chỉnh chuỗi, gấp RNA	Khoảng cách chỉnh sửa với ma trận điểm số sinh học; bài gấp RNA là quy hoạch động trên khoảng. Đây là nơi quy hoạch động chạy ở quy mô lớn nhất trong khoa học.
Xử lý tiếng nói và tín hiệu	Viterbi trong mô hình Markov ẩn, xoắn thời gian động	Cùng thuật toán, khác cách gọi. Xoắn thời gian động là bài hai chuỗi với chi phí căn chỉnh — dùng để so hai chuỗi tín hiệu có tốc độ khác nhau.
Thị giác máy tính, stereo	So khớp ảnh theo dòng quét bằng quy hoạch động	Bài toán tìm độ lệch cho từng điểm ảnh trên một dòng, với ràng buộc tron giữa các điểm kề — đúng khuôn họ một chiều với chi phí chuyển. Cùng bài toán về sau được giải bằng cắt đồ thị ($C/18$), một ví dụ đẹp về hai công cụ cho cùng mô hình.
Tài chính	Định giá quyền chọn kiểu Mỹ bằng cây nhị thức	Đi ngược từ thời điểm đáo hạn về hiện tại, mỗi nút lấy cực đại giữa thực hiện ngay và giữ tiếp — đúng quy hoạch động trò chơi với thứ tự tính ngược thời gian.
Vận trù học	Quản lý tồn kho, lập lịch sản xuất	Bài toán gốc mà Bellman nhắm tới. Ràng buộc thực tế thêm vào là bất định về nhu cầu, dẫn tới quy hoạch động ngẫu nhiên — trạng thái mang theo phân phối thay vì giá trị.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Vì phần khó không nằm ở kỹ thuật mà nằm ở việc *đặt tên cho đúng thứ*. Đệ quy và bảng là chuyện cơ khí; cái khó là nhìn một bài toán và nhận ra “mọi lịch sử dẫn tới đây đều tương đương với nhau nếu ta chỉ nhớ hai con số này”. Đó là một bước trừu tượng hoá, và nó không

có quy trình máy móc. Thêm nữa, cách dạy phổ biến — trình bày công thức đã hoàn chỉnh — che mất chính bước này, khiến người học thấy lời giải hiển nhiên mà không tái tạo được.

- A2. Vì trạng thái thiếu vẫn cho code chạy và vẫn ra số, chỉ là số sai — một lỗi im lặng. Ba cách kiểm tra trước khi cài: thử kể lại tương lai từ trạng thái và xem có phải nói “còn tùy vào trước đó” không; tìm hai lịch sử cùng trạng thái nhưng khác tương lai tối ưu; và kiểm lập luận cắt-dán, vì chỗ nó hỏng chỉ ra đúng chiều còn thiếu. Sau khi cài, cách kiểm rẻ nhất vẫn là stress test với vết cạn trên n nhỏ.
- A3. Vì W xuất hiện trong chi phí như một *giá trị*, trong khi kích thước dữ liệu vào chỉ chứa $\log W$ bit để biểu diễn nó. Với $W = 10^{18}$, dữ liệu vào vẫn ngắn nhưng thuật toán cần 10^{18} bước — tức là mũ theo kích thước biểu diễn. Loại thuật toán này gọi là giả đa thức, và sự tồn tại của nó hoàn toàn nhất quán với việc bài toán là NP-khó (D/23). Hệ quả thực hành: khi trạng thái chứa một giá trị chứ không phải chỉ số, hãy kiểm tra ngay biên của giá trị đó.
- A4. Chọn ghi nhớ khi không gian trạng thái *thưa* — chỉ một phần nhỏ trạng thái thực sự được dùng — hoặc khi thứ tự tính khó xác định, hoặc khi bạn đang trong giai đoạn nghĩ và muốn code gần với suy nghĩ. Chọn điền bảng khi cần tốc độ (không có chi phí gọi hàm), khi độ sâu đệ quy có thể gây tràn ngăn xếp, hoặc khi cần giảm bộ nhớ bằng mảng cuộn — kỹ thuật gần như không áp được cho bản ghi nhớ.
- A5. Còn ba hướng. Thứ nhất, tấn công chi phí mỗi chuyển bằng cấu trúc dữ liệu hoặc bằng các tối ưu ở mục 6: hàng đợi đơn điệu, cây phân đoạn, bao lồi, tính đơn điệu của điểm tối ưu. Thứ hai, giảm số trạng thái bằng một quan sát tham lam — chứng minh rằng cả một lớp lựa chọn không bao giờ tối ưu và loại chúng; hướng này thường hiệu quả nhất. Thứ ba, đổi vai trò các chiều, ví dụ đặt trạng thái theo giá trị thay vì theo khối lượng khi giá trị nhỏ hơn nhiều.

NÓI LẠI THẬT ĐƠN GIẢN

Quy hoạch động chỉ là hai chuyện gộp lại: thử mọi khả năng, và *ghi lại những gì đã tính để khỏi tính lại*.

Hãy nghĩ tới việc leo cầu thang mỗi bước một hoặc hai bậc, và bạn muốn đếm số cách leo lên bậc thứ mười. Nếu ngồi vẽ cây mọi khả năng thì rất nhiều nhánh trùng nhau: dù bạn tới bậc thứ bảy bằng đường nào, số cách đi tiếp từ đó vẫn y hệt. Vậy thay vì vẽ lại, hãy ghi “từ bậc bảy còn bao nhiêu cách” vào một tờ giấy và dùng lại. Số việc từ chỗ tăng gấp đôi mỗi bậc trở thành mỗi bậc một dòng ghi chép.

Câu quan trọng nhất trong chuyện trên là câu “dù bạn tới bậc thứ bảy bằng đường nào”. Đó chính là điều kiện để cách làm này chạy được: những gì đã xảy ra không còn ảnh hưởng gì tới tương lai, ngoài việc bạn đang đứng ở bậc nào. Nếu bài toán có thêm luật kiểu “không được đi hai bước dài liên tiếp” thì đứng ở bậc bảy chưa đủ — bạn còn phải nhớ bước vừa rồi dài hay ngắn. Lúc đó tờ giấy ghi chép phải có hai cột thay vì một.

Toàn bộ khó khăn của quy hoạch động nằm ở đúng chỗ đó: quyết định *tờ giấy cần ghi những gì*. Ghi thiếu thì kết quả sai một cách im lặng; ghi thừa thì tờ giấy to quá và chương trình chạy chậm. Mọi kỹ thuật còn lại trong chương — cách điền bảng, cách tiết kiệm giấy, cách tăng tốc — đều là chuyện phụ so với câu hỏi này.

Một mẹo kiểm tra rất hiệu quả: hãy tự kể lại phần còn lại của bài toán chỉ dựa vào những gì đã ghi. Nếu bạn thấy mình phải nói “còn tùy trước đó tôi làm gì”, thì đúng cái “tùy” đó là thứ còn thiếu.

Một câu: *Trạng thái phải đủ: biết trạng thái thì tương lai không cần biết gì thêm về quá khứ.*

Chỗ không rút gọn được: Việc nghĩ ra trạng thái đúng cho một bài mới vẫn là bước sáng tạo, không có quy trình máy móc — chỉ có các cách kiểm tra sau khi đã đoán.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Quy hoạch động là gì	Vết cạn cộng trí nhớ: các nhánh khác nhau gặp lại cùng bài toán con, nên lưu kết quả để khỏi tính lại. Chi phí bằng số trạng thái nhân chi phí mỗi chuyển.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Quy trình năm bước	Trạng thái → công thức chuyển → cơ sở → thứ tự tính → đáp án và truy vết. Bước một chiếm chín phần mười độ khó.
●●● Tiêu chuẩn của trạng thái tốt	Biết trạng thái thì tương lai hoàn toàn xác định, không cần biết cách tới đó. Đây là tính chất Markov, cũng là một bất biến.
●●● Ba cách phát hiện trạng thái thiếu	Kể lại tương lai và xem có phải nói “còn tuý”; tìm hai lịch sử cùng trạng thái khác tương lai; kiểm lập luận cắt-dán và xem nó hỏng ở đâu.
●●● Sáu họ trạng thái	Một chiều $f[i]$; ba lô $f[i][w]$; hai chuỗi $f[i][j]$; khoảng $f[l][r]$; trên cây $f[u][\cdot]$; mặt nạ bit $f[\text{mask}]$.
●●● Vì sao ba lô vẫn NP-khó	$O(nW)$ là giả đa thức: đa thức theo giá trị W nhưng mũ theo số bit biểu diễn W . Dấu hiệu: trạng thái chứa một giá trị chứ không phải chỉ số.
●● Ghi nhớ so với điền bảng	Ghi nhớ: gần cách nghĩ, chỉ tính trạng thái cần, không phải tìm thứ tự. Điền bảng: nhanh hơn, không tràn ngăn xếp, cho phép mảng cuộn.
●● Mảng cuộn	Giữ hai tầng (hoặc một, nếu duyệt đúng chiều) khi tầng i chỉ phụ thuộc tầng $i - 1$; cái giá là mất khả năng truy vết.
●● Bốn kỹ thuật tăng tốc chuyển	Deque đơn điệu (cửa sổ trượt); cây phân đoạn/Fenwick (đoạn tùy ý); bao lồi (chuyển dạng đường thẳng); tính đơn điệu điểm tối ưu (chia để trị, Knuth).
●● Hướng tăng tốc mạnh nhất	Giảm số trạng thái bằng một quan sát tham lam: chứng minh cả một lớp lựa chọn không bao giờ tối ưu rồi loại.
●● Mẹo đảo vai trò	Khi W quá lớn nhưng tổng giá trị nhỏ, đặt trạng thái theo giá trị và tối thiểu hoá khối lượng.
●● Ba họ chuyên hơn	Quy hoạch động trên chữ số (đếm số thoả điều kiện); kỳ vọng (có thể phải giải hệ); trò chơi (cực đại và cực tiểu xen kẽ).
● Truy vết lời giải	Lưu quyết định tại mỗi trạng thái, hoặc dò ngược bằng so sánh giá trị; nếu đã dùng mảng cuộn thì cần kỹ thuật chia để trị kiểu Hirschberg.
●● Bất đẳng thức tứ giác	$w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với $a \leq b \leq c \leq d$ — “lông nhau không đắt hơn cắt nhau” — kéo theo điểm chia tối ưu đơn điệu, tức giảm được một thừa số n . Kiểm bằng vết cạnh nhỏ trước khi tin.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi luôn bí ở bước đặt trạng thái. Có quy trình nào không?

| Có một quy trình bốn bước dùng được cho phần lớn bài. Một, viết lời giải vết cạnh để quy trước, với các tham số tự nhiên. Hai, nhìn danh sách tham số của hàm đệ quy đó — chúng chính là ứng viên trạng thái. Ba, loại các tham số suy ra được từ những tham số khác (ví dụ nếu biết i thì biết luôn tổng đã dùng thì bỏ tham số tổng). Bốn, kiểm tra tính đủ bằng ba câu hỏi ở khối *Cần nhớ*. Quy trình này biến việc “nghĩ ra trạng thái” thành việc “viết vết cạnh rồi rút gọn tham số”, dễ hơn nhiều.

2. Làm sao biết bài này nên dùng tham lam hay quy hoạch động?

| Thử tham lam trước vì nó rẻ hơn nhiều: nghĩ một tiêu chí, và stress test với vết cạnh trên n nhỏ trong năm phút. Nếu tìm được phản ví dụ thì chuyển sang quy hoạch động, và điều đáng chú ý là chính phản ví dụ đó thường chỉ ra chiều trạng thái cần thêm — nó cho thấy quyết định tối ưu phụ thuộc vào cái gì. Đây là một trong những cách hiệu quả nhất để “đọc ra” trạng thái từ bài toán.

3. Số trạng thái của tôi là 10^7 và mỗi chuyển $O(1)$. Có chạy nổi không?

| Theo bảng ở A/03 thì 10^7 phép toán đơn giản là thoải mái, nhưng hãy kiểm hai điều mà bảng không nói. Thứ nhất là bộ nhớ: 10^7 số nguyên 32 bit là 40 MB, thường vẫn được, nhưng 10^7 số 64 bit là 80 MB và có thể vượt giới hạn. Thứ hai là kiểu truy cập: nếu bạn duyệt bảng hai chiều theo cột thay

vì theo hàng thì mỗi lần truy cập là một lần trượt cache và chương trình có thể chậm gấp năm lần. Thứ tự vòng lặp là tối ưu rẻ nhất và hay bị bỏ qua nhất.

4. Quy hoạch động của tôi có chu trình phụ thuộc — trạng thái A cần B mà B lại cần A. Làm sao?

| Đó là dấu hiệu bài toán không còn là quy hoạch động thuần túy. Ba hướng xử lý. Nếu các chuyển có “chi phí không âm” và bạn tìm cực tiểu, hãy dùng Dijkstra trên đồ thị trạng thái (C/17) — thứ tự tính được xác định động thay vì tĩnh. Nếu là bài kỳ vọng có chu trình, hãy lập và giải hệ phương trình tuyến tính. Nếu chu trình đến từ việc trạng thái thiếu chiều, hãy thêm chiều để phá chu trình — ví dụ thêm chiều “số bước đã đi”.

5. Tôi cần in ra lời giải cụ thể chứ không chỉ giá trị tối ưu. Làm thế nào?

| Hai cách. Cách đơn giản: lưu thêm mảng `choice[state]` ghi quyết định đã chọn tại mỗi trạng thái, rồi đi ngược từ trạng thái đáp án. Cách tiết kiệm bộ nhớ: không lưu gì, và khi truy vết thì tính lại — tại mỗi trạng thái, thử từng quyết định và xem quyết định nào cho ra đúng giá trị đã lưu. Nếu bạn đã dùng mảng cuộn thì không truy vết trực tiếp được, và cách chuẩn cho họ hai chuỗi là kỹ thuật chia đôi của Hirschberg, cho $O(nm)$ thời gian với $O(\min(n, m))$ bộ nhớ.

6. Vì sao dãy con tăng dài nhất lại có lời giải $O(n \log n)$ trông chẳng giống quy hoạch động?

| Vì nó là quy hoạch động đã được biến đổi. Bản $O(n^2)$ có trạng thái “độ dài dãy tăng dài nhất kết thúc tại i ”. Bản $O(n \log n)$ đảo lại: với mỗi độ dài, giữ *giá trị kết thúc nhỏ nhất* có thể. Mảng các giá trị đó luôn tăng, nên cập nhật được bằng tìm kiếm nhị phân. Đây là ví dụ điển hình của mẹo đảo vai trò ở mục 4 — đổi chỗ giữa cái được tối ưu và cái làm chỉ số — và nó đáng học vì mẹo này áp dụng được cho nhiều bài khác.

7. Quy hoạch động trên cây khác gì so với trên dãy?

| Khác chủ yếu ở thứ tự tính và ở cách gộp. Thứ tự là duyệt sâu, tính cây con trước rồi gộp lên cha — tương đương với việc “từ dưới lên” trên một cấu trúc phân nhánh. Cách gộp thường có dạng “kết hợp kết quả của các con”, và khi có ràng buộc về số lượng thì phép gộp giống bài ba lô trên từng cây con. Một chi tiết kỹ thuật quan trọng: với cây sâu tới 10^5 , đệ quy có thể tràn ngăn xếp, nên hãy dùng bản lập hoặc tăng giới hạn ngăn xếp (E/25).

8. Có bài nào nhìn không giống quy hoạch động mà thực ra là không?

| Rất nhiều, và nhận ra chúng là dấu hiệu đã nắm phương pháp. Ba ví dụ: bài đếm đường đi trên lưới có vật cản (thực ra là $f[i][j]$ một cách hiển nhiên); bài chia dãy thành k đoạn tối ưu (họ khoảng, nhưng thường phải kèm tối ưu chia để trị); và bài xác suất thắng của một trò chơi (quy hoạch động kỳ vọng, với chú ý về chu trình). Ngoài ra, mọi bài tìm đường đi ngắn nhất trên đồ thị không chu trình đều là quy hoạch động — và ngược lại, mọi quy hoạch động đều là bài đường đi ngắn nhất trên đồ thị trạng thái, một cách nhìn rất đáng có (C/17).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Số cách leo cầu thang, số cách đi trên lưới có vật cản	AtCoder DP “Frog 1”; CSES “Grid Paths”[5]	Bài nhập môn; mục tiêu là viết đủ năm bước ra giấy trước khi gõ.
● Đổi tiền: số cách và số đồng ít nhất	CSES “Coin Combinations I/II”[5]	Hai bài khác nhau đúng ở thứ tự hai vòng lặp — hãy giải thích vì sao.
● Dãy con tăng dài nhất	CSES “Increasing Subsequence”[5]	Cài cả $O(n^2)$ và $O(n \log n)$; hiểu mẹo đảo vai trò.
●● Ba lô 0/1 và ba lô không giới hạn	CSES “Book Shop”[5]	Giảm bộ nhớ bằng mảng cuộn; chú ý chiều duyệt w cho hai biến thể.
●● Khoảng cách chỉnh sửa	CSES “Edit Distance”[5]	Họ hai chuỗi; sau đó thử truy vết ra dãy thao tác cụ thể.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Dây con chung dài nhất của hai chuỗi	Bài kinh điển	Cùng họ; so sánh cách truy vết với và không có mảng cuộn.
•• Tập độc lập lớn nhất trên cây	Bài kinh điển	Quy hoạch động trên cây với hai trạng thái mỗi đỉnh; chú ý độ sâu đệ quy.
•• Cắt gậy để tổng chi phí nhỏ nhất	Bài kinh điển	Họ khoảng; nhận ra sự tương đồng với bài nhân dây ma trộn.
••• Người bán hàng với $n \leq 20$	CSES “Hamiltonian Flights”-loại[5]	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
••• Chia dây thành k đoạn sao cho tổng chi phí nhỏ nhất	Bài thi đấu kinh điển	Bản $O(kn^2)$ trước, rồi thêm tối ưu chia để trị; kiểm tính đơn điệu điểm tối ưu bằng thực nghiệm.
••• Đếm số nguyên trong $[a, b]$ thoả một điều kiện về chữ số	Bài kinh điển	Quy hoạch động trên chữ số với cờ “đang bám biên”; đây là họ có nhiều trường hợp biên nhất.
••• Xác suất thắng của một trò chơi có chu trình trạng thái	Bài kinh điển	Lập hệ phương trình tuyến tính; ví dụ cho thấy ranh giới của quy hoạch động thuần túy.
••• Ghép cặp hoàn hảo trọng số nhỏ nhất với $n \leq 20$	Bài kinh điển	Mặt nạ bit; so với thuật toán Hungarian ở $C/18$ về cả độ phức tạp lẫn độ dài code.
••• Bài toán ba lô với W tới 10^{18} nhưng tổng giá trị nhỏ	Bài kinh điển	Đảo vai trò hai chiều; bài mẫu cho mẹo ở mục 4.

Kết chương và đọc tiếp

Quy hoạch động là kỹ thuật có tỉ lệ xuất hiện cao nhất trong mọi kỳ thi và phỏng vấn, và toàn bộ độ khó của nó tập trung vào một câu: trạng thái phải đủ. Ba điều đáng mang theo: quy trình năm bước viết ra thành lời trước khi gõ; ba cách kiểm tra tính đủ của trạng thái; và cách nhìn “chi phí bằng số trạng thái nhân chi phí chuyển”, vì nó chỉ ra ngay phải tấn công thừa số nào khi quá chậm.

Chương tiếp theo đổi ngôn ngữ. Rất nhiều bài toán trông rất khác nhau trở thành cùng một bài khi được phát biểu lại bằng đỉnh và cạnh — và như câu hỏi cuối ở mục hỏi đáp đã gợi ý, quy hoạch động và đường đi ngắn nhất là hai cách nói về cùng một thứ. Đồ thị là chương của kỹ năng mô hình hoá, kỹ năng đáng giá hơn việc thuộc thêm mười thuật toán ($C/17$).

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Yao, “Speed-Up in Dynamic Programming”	SIAM Journal on Algebraic and Discrete Methods, 1982	Nguồn của điều kiện tứ giác ở dạng tổng quát. Đọc phần phát biểu định lý và ví dụ; đây là bài nên đọc nếu bạn hay gặp quy hoạch động trên khoảng.
nền tảng	Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences”	Communications of the ACM, 1975	Ba trang. Đọc để có kỹ thuật truy vết trong bộ nhớ tuyến tính — thứ mà mảng cuộn lấy đi.
tra khi cần	Knuth, “Optimum Binary Search Trees”	Acta Informatica, 1971	Nơi tính đơn điệu của điểm chia xuất hiện lần đầu, trong một bài toán cụ thể.
tra khi cần	Aggarwal, Klawe, Moran, Shor, Wilber, “Geometric Applications of a Matrix-Searching Algorithm”	Algorithmica, 1987	Thuật toán SMAWK. Đọc khi truy hồi của bạn viết được thành “cực tiểu theo hàng của một ma trận đơn điệu toàn phần”.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc khi bạn tự hỏi vì sao khoảng cách chỉnh sửa mãi vẫn là $O(nm)$: có lý do, và nó nằm ở D/24.

Nguồn

- [1] D. E. Knuth, "Optimum Binary Search Trees," *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [2] F. F. Yao, "Speed-Up in Dynamic Programming," *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [3] A. Aggarwal, M. M. Klawe, S. Moran, et al., "Geometric Applications of a Matrix-Searching Algorithm," *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [4] D. S. Hirschberg, "A Linear Space Algorithm for Computing Maximal Common Subsequences," *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [5] A. Laaksonen. "CSES Problem Set." (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).